

Relatório de Atividades – Unidade 1

IA Generativa – IMD3004 – 2026.2

Participantes:

- ERICKSON LINS LIMEIRA - Matrícula 20260039784
- GIL CÉSAR FARIA - Matrícula 20250082520

Data de Entrega:

- 28/09/2026

Sumário

SEÇÃO 1 – GAN	2
1.1 – SETUP	2
1.2 – RESULTADOS	3
1.3 – ANÁLISE	5
SEÇÃO 2 – DCGAN	6
2.1 – SETUP	6
2.2 – RESULTADOS	8
2.3 – ANÁLISE	13
SEÇÃO 3 – WGAN CONDICIONAL	14
3.1 – SETUP	14
3.2 – RESULTADOS	14
3.3 – ANÁLISE	18
SEÇÃO 4 – PRECISÃO E REVOCAÇÃO DE VARIEDADES	19
4.1 – SETUP	19
4.2 – RESULTADOS	20
4.3 – ANÁLISE	22
SEÇÃO 5 – MODELOS AUTORREGRESSIVOS COM RNN	23
5.1 – SETUP	23
5.2 – RESULTADOS	23
5.3 – ANÁLISE	26
SEÇÃO 6 – MINI GPT	27
6.1 – SETUP	27
6.2 – RESULTADOS	28
6.3 – ANÁLISE	29

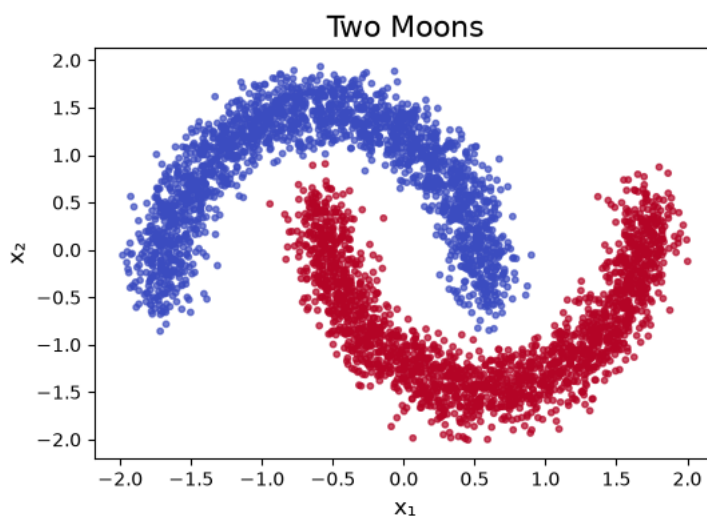
Seção 1 – GAN

1.1 – Setup

1.1.1 Dados e normalização

Foi utilizado o conjunto sintético **Two Moons**, gerado com `make_moons` do `scikit-learn` utilizando `n_samples=4096`, `noise=0.1` e `random_state=42`. O conjunto é formado por pontos bidimensionais distribuídos em duas regiões curvas intercaladas, o que permite avaliar visualmente se o gerador consegue aprender uma distribuição não linear em duas dimensões.

Antes do treinamento, os dados foram normalizados com `StandardScaler`, de forma que cada dimensão passasse a apresentar média aproximadamente igual a zero e desvio padrão unitário. Essa transformação foi aplicada aos pontos reais utilizados no treinamento e preservada como referência para a comparação com as amostras produzidas pelo gerador. O gráfico a seguir mostra o conjunto de dados gerado.



1.1.2 Arquiteturas do gerador e discriminador

O gerador foi implementado como uma rede totalmente conectada que recebe um vetor de ruído bidimensional e produz diretamente um ponto com duas coordenadas, compatível com o espaço dos dados **Two Moons**. A arquitetura utilizada foi $2 \rightarrow 16 \rightarrow 32 \rightarrow 2$, com ativações **ReLU** nas camadas ocultas e saída linear na última camada.

O discriminador também foi implementado como uma rede totalmente conectada, recebendo pontos bidimensionais e produzindo uma probabilidade escalar de pertencimento à distribuição real. Sua arquitetura foi $2 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 1$, utilizando **ReLU** nas camadas ocultas, `Dropout(0,3)` após cada uma delas e ativação **Sigmoid** na saída.

As arquiteturas foram adaptadas ao caráter bidimensional do problema. Diferentemente de tarefas com imagens, não são necessárias camadas convolucionais nem operações de redimensionamento espacial, já que tanto as amostras reais quanto as geradas são representadas diretamente por vetores de duas dimensões.

1.1.3 Hiperparâmetros e treinamento

O treinamento foi realizado por **1.000 épocas**, sem parada antecipada, conforme especificado. Foi utilizado `batch_size=128`, vetor de ruído com dimensão `z_dim=2` amostrado de uma distribuição normal padrão e `BCELoss` como função de perda para gerador e discriminador.

As duas redes foram otimizadas com **Adam**, utilizando taxa de aprendizado 0,0002 e parâmetros `betas=(0,5, 0,999)`. Em cada lote, o discriminador foi atualizado utilizando amostras reais rotuladas como 1 e amostras produzidas pelo gerador rotuladas como 0. Em

seguida, o gerador foi atualizado buscando aumentar a probabilidade de suas amostras serem classificadas pelo discriminador como reais.

Durante o treinamento, foram acumuladas as perdas médias do gerador e do discriminador em cada época. Também foram registradas amostras produzidas pelo gerador ao longo do processo, permitindo acompanhar visualmente a evolução da distribuição aprendida em diferentes momentos do treinamento.

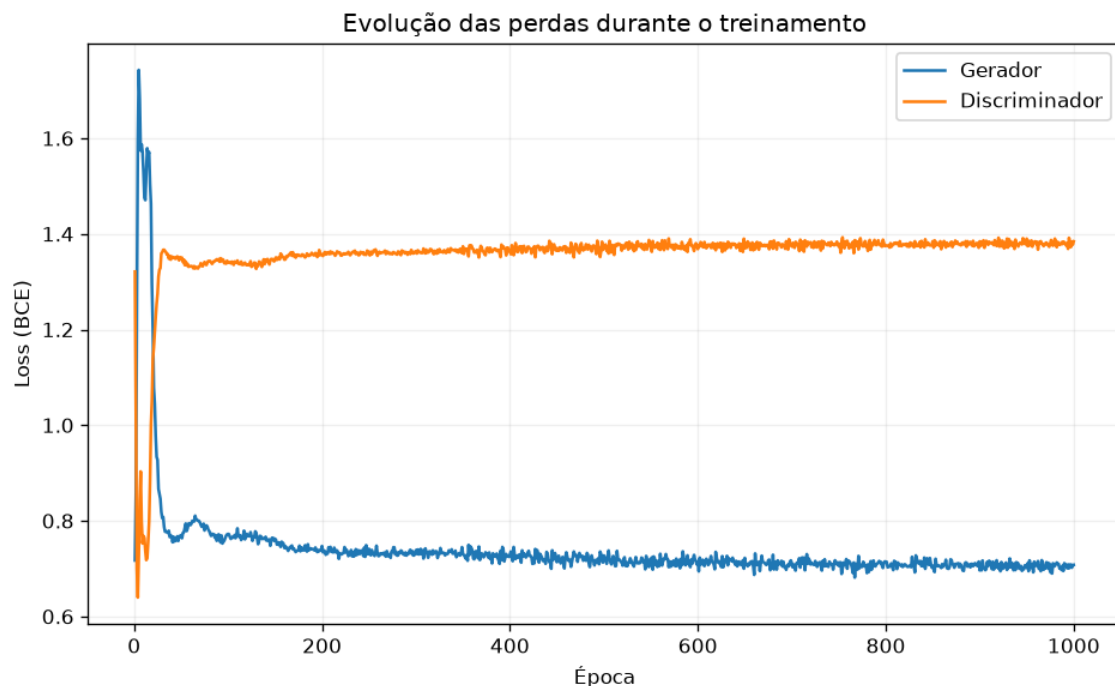
1.2 – Resultados

1.2.1 Curvas de perda

As perdas médias do gerador e do discriminador foram registradas ao longo das 1.000 épocas de treinamento. Nas primeiras épocas, a perda do gerador permanece próxima de 0,72, enquanto a perda acumulada do discriminador fica em torno de 1,32. Ao longo do treinamento, ambas apresentam oscilações moderadas, mas sem comportamento explosivo ou divergente.

A perda do gerador inicialmente aumenta, atingindo valores próximos de 0,77 por volta das épocas 50–100, e depois passa a diminuir gradualmente, terminando em 0,7079 na época 1.000. A perda do discriminador apresenta trajetória complementar: parte de 1,3211, cresce lentamente e termina em 1,3844.

Nas etapas finais, a perda do gerador permanece próxima de 0,69 e a perda acumulada do discriminador próxima de 1,38. Esses valores são referências importantes porque, com `BCELoss`, um discriminador que atribui probabilidade próxima de 0,5 tanto para amostras reais quanto para falsas produz aproximadamente $-\log(0,5) \approx 0,693$ de perda em cada uma dessas duas parcelas. Como a perda do discriminador soma as contribuições de reais e falsas, o valor de referência fica próximo de $2 \times 0,693 \approx 1,386$. Já o gerador, ao tentar fazer suas amostras serem classificadas como reais, também apresenta perda próxima de 0,693 quando o discriminador responde em torno de 0,5. Portanto, aproximar **Loss G $\approx$ 0,69** e **Loss D $\approx$ 1,38** sugere um estado em que o discriminador já não consegue separar com facilidade as duas distribuições.



1.2.2 Evolução das amostras geradas

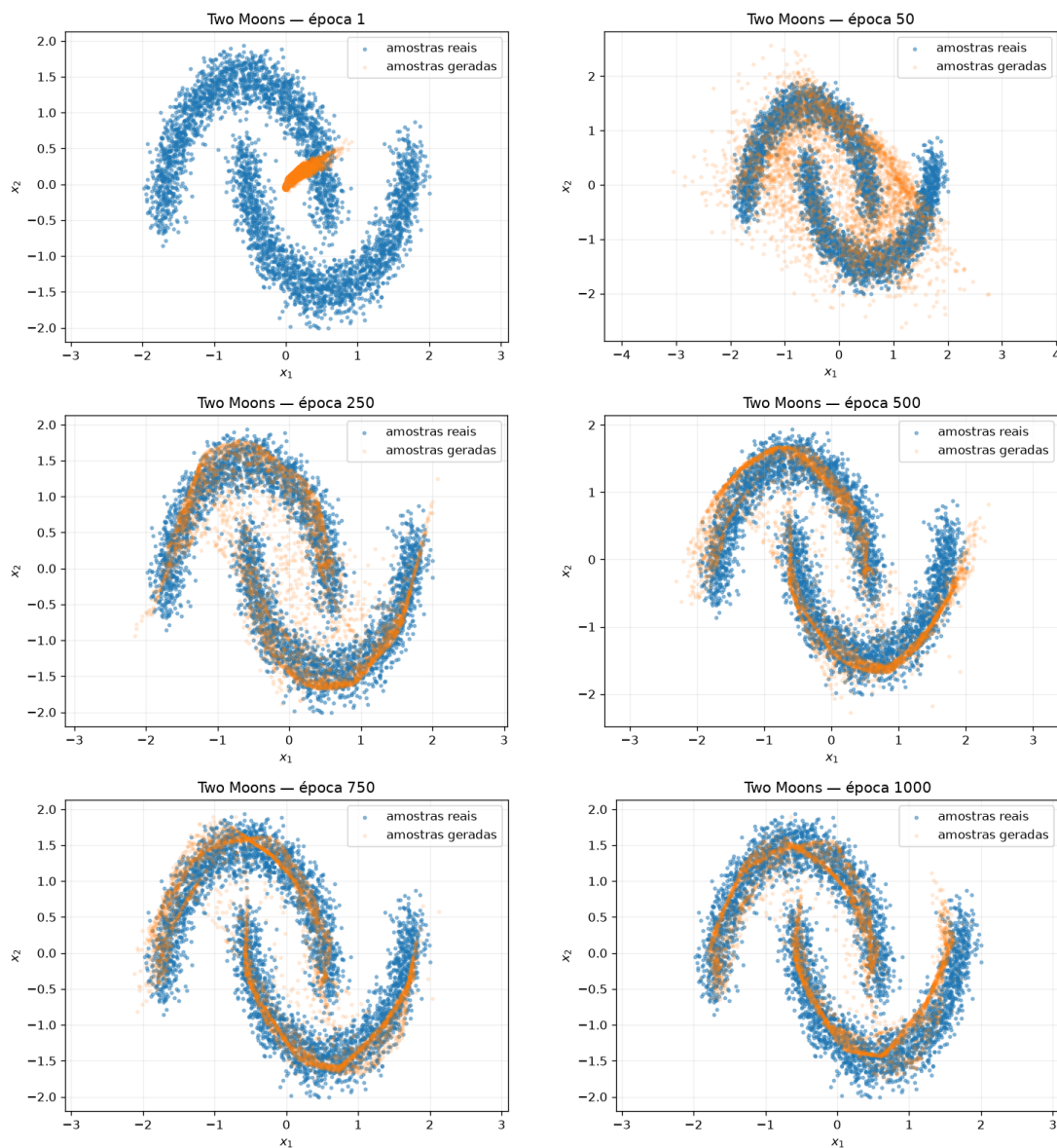
A evolução das amostras geradas foi acompanhada em seis momentos do treinamento, nas épocas 1, 50, 250, 500, 750 e 1000, permitindo observar de forma mais gradual como o gerador passou a aproximar a distribuição real do **Two Moons**.

Na época 1, as amostras permanecem concentradas em uma pequena região e ainda não apresentam relação clara com a geometria dos dados reais. Na época 50, a distribuição gerada começa a se expandir, indicando o início da aprendizagem da estrutura global, embora as duas meias-luas ainda não estejam adequadamente representadas.

Na época 250, as amostras já passam a ocupar regiões próximas às curvas reais e a separação entre as duas estruturas começa a surgir. Na época 500, a geometria das duas meias-luas torna-se claramente reconhecível e a distribuição gerada cobre uma parcela maior do suporte dos dados reais.

Nas épocas 750 e 1000, o gerador reproduz de forma consistente o traçado global das duas meias-luas e alcança praticamente toda a extensão das curvas. Entretanto, observa-se uma diferença importante em relação à distribuição real: as amostras geradas ficam excessivamente concentradas em uma faixa estreita próxima ao centro de cada meia-lua, formando visualmente uma trajetória semelhante a uma linha.

Os dados reais, por efeito do ruído utilizado em `make\_moons`, apresentam uma dispersão transversal consideravelmente maior ao redor dessas curvas. Assim, o gerador aprendeu adequadamente a geometria e a localização dos principais modos da distribuição, mas subestimou sua variabilidade local, produzindo amostras menos dispersas que os dados reais.

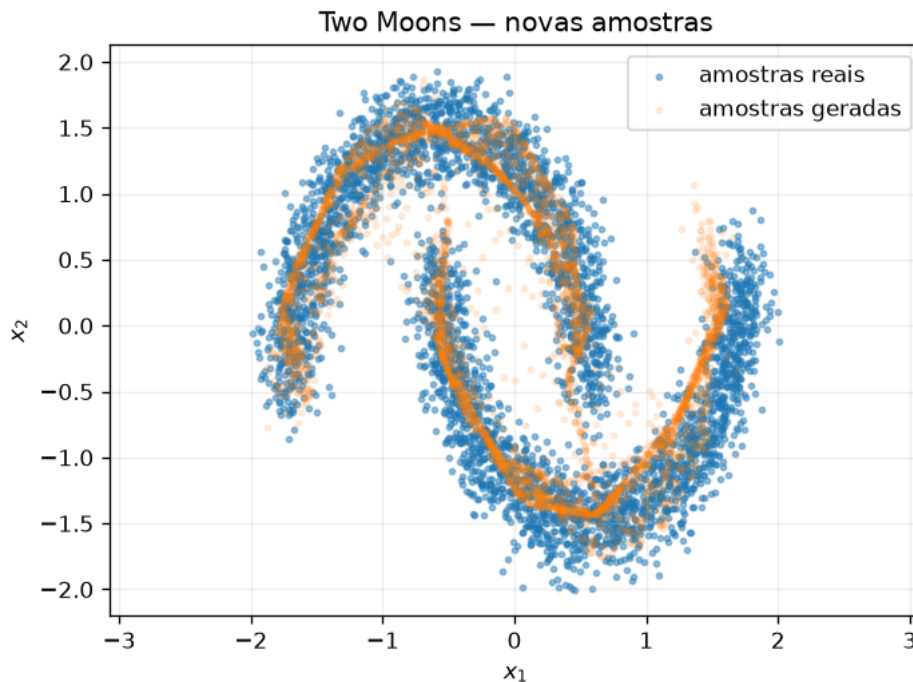


1.2.3 Modelo salvo e recarregado

Ao final do treinamento, o gerador foi salvo no formato **TorchScript** no arquivo `twomoons.pt`. Em seguida, o modelo foi carregado novamente a partir desse arquivo e utilizado para gerar um novo conjunto de amostras a partir de vetores de ruído independentes dos utilizados durante o treinamento.

As amostras produzidas pelo modelo recarregado preservam o mesmo comportamento observado ao final das 1.000 épocas: os pontos acompanham a geometria das duas meias-luas e cobrem suas principais regiões, confirmando que o modelo salvo preserva os parâmetros aprendidos durante o treinamento. Também permanece visível a concentração das amostras em uma faixa mais estreita que a distribuição real, característica já observada no modelo antes de ser salvo.

Esse teste confirma que o gerador pode ser persistido, carregado posteriormente e utilizado para produzir novas amostras sem necessidade de novo treinamento.



1.3 – Análise

A inspeção visual mostra que o gerador conseguiu aprender a geometria global do conjunto **Two Moons**, reproduzindo as duas estruturas curvas e sua disposição relativa no espaço. Entretanto, nas etapas finais, as amostras geradas permanecem excessivamente concentradas em uma faixa estreita ao longo das meias-luas, apresentando menor dispersão transversal do que os dados reais e, portanto, subestimando parte da variabilidade introduzida pelo ruído do conjunto original.

As curvas de perda sugerem um treinamento estável, sem sinais evidentes de divergência. Ao final, Loss_G permanece próxima de 0,69 e Loss_D próxima de 1,38, valores compatíveis com um discriminador que atribui probabilidades próximas de 0,5 para amostras reais e geradas. Em conjunto, os resultados indicam que a GAN aprendeu adequadamente a estrutura principal da distribuição, embora ainda não tenha reproduzido completamente sua dispersão local.

Seção 2 – DCGAN

2.1 – Setup

2.1.1 Dados e hiperparâmetros de treinamento

Foi utilizado o conjunto **MNIST**, composto por imagens em escala de cinza de dimensão 28×28 , distribuídas entre as 10 classes correspondentes aos dígitos de 0 a 9. As imagens foram convertidas para tensores e normalizadas do intervalo $[0, 1]$ para $[-1, 1]$ pela transformação $x' = 2x - 1$, compatível com a ativação “**tanh**” utilizada na saída do gerador.

Para manter os experimentos comparáveis, foram utilizados em todas as configurações “**SEED=42**”, “**batch_size=32**”, vetor latente de dimensão “**z_dim=100**” e treinamento por **10 épocas**. O vetor latente foi amostrado de uma distribuição normal padrão. A mesma semente também foi utilizada na geração dos vetores fixos empregados posteriormente na comparação da evolução dos modelos, das amostras finais e das interpolações.

Na configuração base, o treinamento adversarial utilizou **BCELoss** como função de perda e **RMSprop** como otimizador tanto para o gerador quanto para o discriminador, com taxa de aprendizado de 10^{-3} para ambas as redes. A cada lote, o discriminador foi atualizado utilizando amostras reais rotuladas como 1 e amostras produzidas pelo gerador rotuladas como 0. Em seguida, o gerador foi atualizado buscando fazer com que suas amostras fossem classificadas pelo discriminador como reais.

2.1.2 Arquiteturas da configuração base

O gerador da configuração base recebe um vetor latente **z** de dimensão **100** e o projeta inicialmente para uma representação com **128** mapas de características de dimensão 7×7 . Em seguida, essa representação passa por camadas convolucionais transpostas que aumentam progressivamente a resolução espacial de 7×7 para 14×14 e, posteriormente, para 28×28 , reduzindo os canais intermediários de 128 para 64 até produzir uma imagem com um único canal. Foram utilizadas normalização em lote e ativações não lineares nas camadas intermediárias, enquanto a camada de saída utiliza **tanh**, produzindo valores no intervalo $[-1, 1]$.

O discriminador realiza o processo inverso. A imagem de entrada $1 \times 28 \times 28$ é processada por camadas convolucionais que reduzem progressivamente a resolução para 14×14 e 7×7 , utilizando inicialmente 64 e depois 128 canais. As camadas intermediárias empregam **LeakyReLU** e **Dropout(0,4)**. Após o processamento convolucional, as características são achatadas e encaminhadas para uma camada linear com saída sigmoide, que produz a estimativa de probabilidade de a imagem recebida pertencer ao conjunto de dados real.

2.1.3 Configurações experimentais

Foram executadas dez configurações experimentais. A configuração base foi utilizada como referência e, nos demais experimentos, apenas o fator sob investigação foi alterado, mantendo **SEED=42**, **batch_size=32**, **z_dim=100**, **BCELoss** e 10 épocas de treinamento.

Nos experimentos de capacidade, foram avaliados um gerador reduzido, um gerador ampliado, um discriminador reduzido, um discriminador ampliado e uma configuração com gerador e discriminador ampliados simultaneamente. O gerador reduzido utiliza apenas a transição espacial $7 \rightarrow 28$, com 32 canais intermediários, enquanto o ampliado mantém as resoluções $7 \rightarrow 14 \rightarrow 28$, aumenta os canais para 256 e 128 e adiciona duas camadas extras em cada estágio. No discriminador reduzido, os canais foram diminuídos de 64/128 para 24/64. No discriminador ampliado, foram utilizados 80/160 canais e duas camadas extras em cada estágio.

Também foram testadas três combinações de taxas de aprendizado, sempre com RMSprop e com as arquiteturas base. No experimento 6, “lr_gerador_maior”, foram utilizadas taxas de 10^{-3} para o gerador e 10^{-5} para o discriminador. Na configuração 7, “lr_discriminador_maior”, foram utilizadas taxas de 10^{-5} para o gerador e 10^{-3} para o discriminador. Na configuração 8, “lr_ambos_menores”, ambos utilizaram 10^{-4} . Por fim, foi realizado um experimento substituindo RMSprop por SGD, mantendo as taxas de aprendizado em 10^{-3} para as duas redes.

A tabela da página seguinte apresenta de forma resumida todas as variações de experimentos executados.

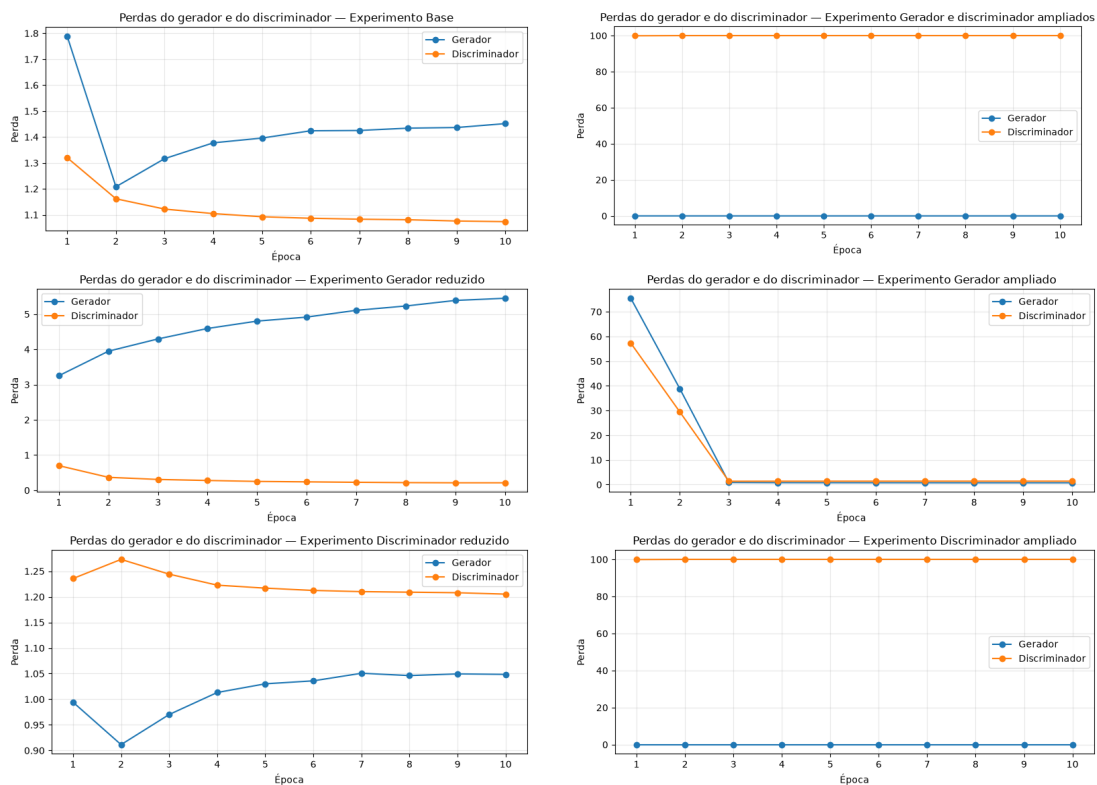
	Experimento	Fator	Gerador	Parâm G	Discriminador	Parâms D	Otimizador	LR G	LR D
0	base	referencia	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻³	10 ⁻³
1	gerador_reduzido	capacidade_gerador	res= 7→28; canais=32; extras=0	159.233	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻³	10 ⁻³
2	gerador_ampliado	capacidade_gerador	res= 7→14→28; canais= 256→128; extras= 2→2	3.567.105	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻³	10 ⁻³
3	discriminador_reduzido	capacidade_discriminador	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 24→64; extras= 0→0	42.225	RMSprop	10 ⁻³	10 ⁻³
4	discriminador_ampliado	capacidade_discriminador	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 80→160; extras= 2→2	906.561	RMSprop	10 ⁻³	10 ⁻³
5	ambos_ampliados	capacidade_ambos	res= 7→14→28; canais= 256→128; extras= 2→2	3.567.105	res= 28→14→7; canais= 80→160; extras= 2→2	906.561	RMSprop	10 ⁻³	10 ⁻³
6	lr_gerador_maior	learning_rate	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻³	10 ⁻⁵
7	lr_discriminador_maior	learning_rate	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻⁵	10 ⁻³
8	lr_ambos_menores	learning_rate	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	RMSprop	10 ⁻⁴	10 ⁻⁴
9	sgd	otimizador	res= 7→14→28; canais= 128→64; extras= 0→0	840.321	res= 28→14→7; canais= 64→128; extras= 0→0	212.865	SGD	10 ⁻³	10 ⁻³

2.2 – Resultados

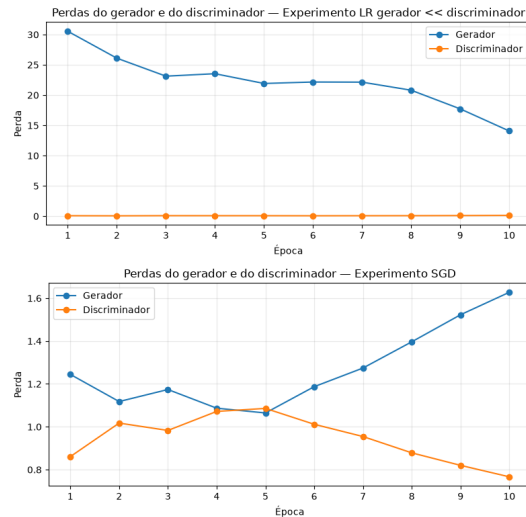
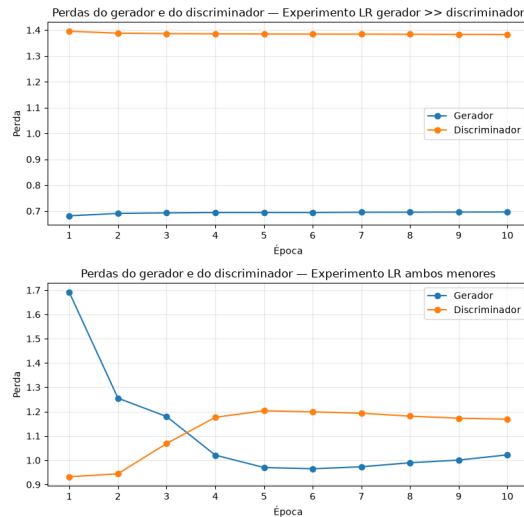
2.2.1 Curvas de perda

As curvas de perda do gerador e do discriminador foram registradas ao longo das 10 épocas para todas as configurações. Como o treinamento é adversarial, as perdas devem ser interpretadas em conjunto: valores baixos isoladamente não indicam necessariamente melhor qualidade das amostras, pois o comportamento relativo das duas redes é mais informativo sobre o equilíbrio do treinamento.

Na configuração base, as perdas permanecem em uma faixa relativamente equilibrada ao longo das épocas, servindo como referência para as demais comparações. As alterações de capacidade produzem comportamentos bastante distintos. O gerador reduzido apresenta crescimento progressivo da perda do gerador e redução da perda do discriminador, indicando vantagem crescente do discriminador. Já os experimentos com discriminador ampliado e com ambas as redes ampliadas evoluem para um regime degenerado, no qual a perda do gerador se aproxima de zero enquanto a perda do discriminador assume valores muito elevados.



As variações das taxas de aprendizado também modificam de forma clara a dinâmica adversarial. Quando a taxa do gerador é muito menor que a do discriminador, a perda do gerador cresce acentuadamente enquanto a perda do discriminador permanece baixa, caracterizando forte desequilíbrio em favor do discriminador. Quando a taxa do gerador é maior que a do discriminador, o comportamento é mais equilibrado. A redução conjunta das duas taxas produz curvas mais suaves e com variações menos abruptas ao longo das épocas.



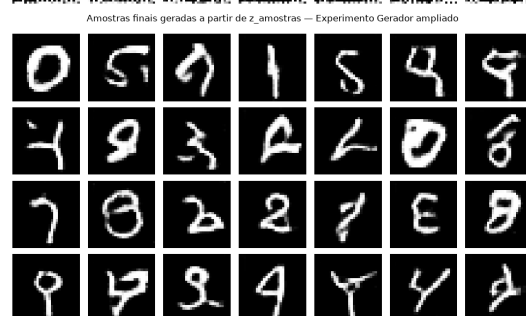
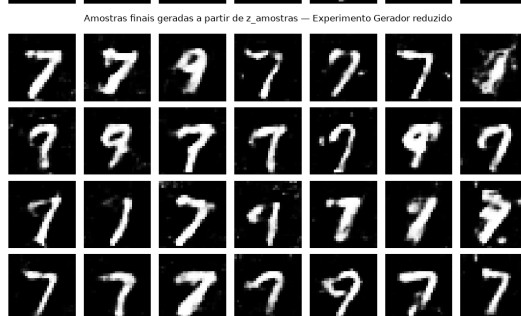
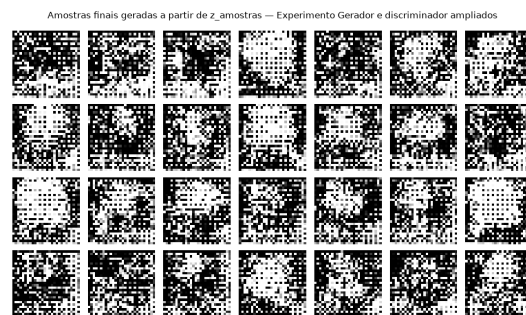
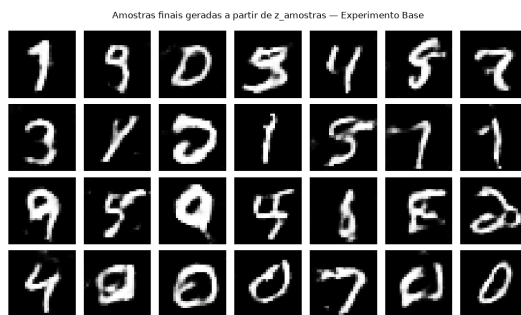
A substituição de RMSprop por SGD também altera o perfil de convergência em relação à configuração base, mostrando que o otimizador exerce influência direta sobre a dinâmica entre gerador e discriminador.

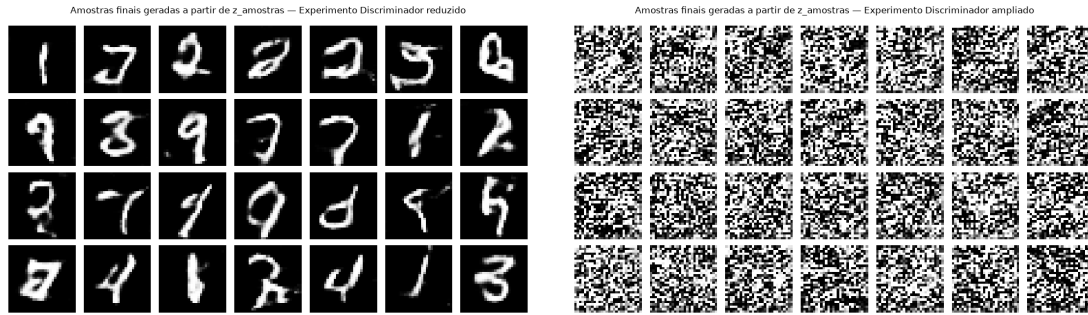
2.2.2 Amostras geradas ao final do treinamento

As amostras finais foram geradas a partir do mesmo conjunto de vetores latentes para todos os experimentos, permitindo comparar diretamente o efeito de cada alteração sobre a qualidade e a diversidade das imagens produzidas.

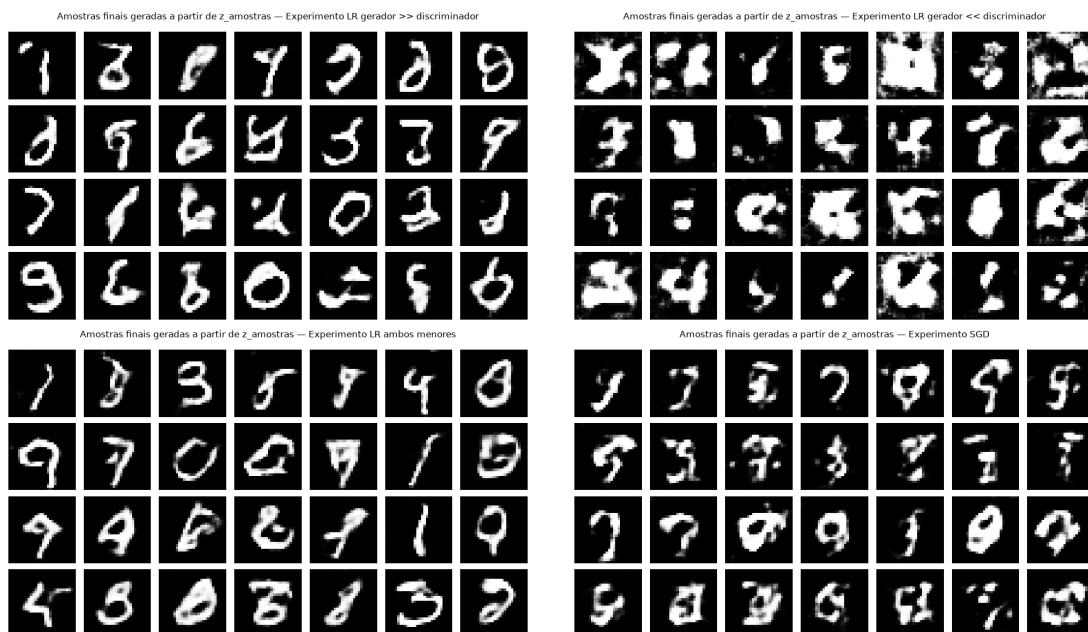
A configuração base produz diferentes dígitos reconhecíveis, embora ainda apresente variações de qualidade entre as amostras. A redução da capacidade do gerador provoca uma perda expressiva de diversidade: as imagens convergem predominantemente para formas semelhantes ao dígito 7, caracterizando um caso evidente de **mode collapse**. O gerador ampliado preserva maior diversidade e produz alguns dígitos reconhecíveis.

As alterações na capacidade do discriminador também apresentam efeitos marcantes. O discriminador reduzido ainda permite a geração de diferentes dígitos com estrutura reconhecível. Em contraste, o discriminador ampliado e a configuração em que gerador e discriminador foram ampliados simultaneamente não produzem dígitos adequadamente estruturados ao final das 10 épocas, permanecendo predominantemente em padrões semelhantes a ruído.





Nos experimentos com taxas de aprendizado, o caso em que a taxa do gerador é muito menor que a do discriminador apresenta amostras bastante degradadas, consistente com o desequilíbrio observado nas curvas de perda. Quando a taxa do gerador é maior que a do discriminador e quando ambas as taxas são reduzidas, são obtidas amostras visualmente mais estruturadas e diversificadas. O experimento com SGD também consegue formar dígitos reconhecíveis, mas apresenta diferenças de qualidade e definição em relação às configurações treinadas com RMSprop.



2.2.3 Evolução com vetor latente fixo

Para acompanhar a evolução qualitativa dos modelos durante o treinamento, foi mantido fixo o mesmo vetor latente e sua saída foi registrada em diferentes épocas. Esse procedimento permite observar como uma mesma posição do espaço latente é transformada pelo gerador ao longo do processo de aprendizagem, evitando que diferenças entre imagens sejam atribuídas apenas à amostragem de vetores distintos.

Na configuração base, a saída parte de uma estrutura pouco definida e passa gradualmente a assumir características compatíveis com um dígito do **MNIST**. Comportamento semelhante é observado nas configurações que mantêm treinamento estável, ainda que com diferenças na velocidade e na qualidade da formação da imagem.

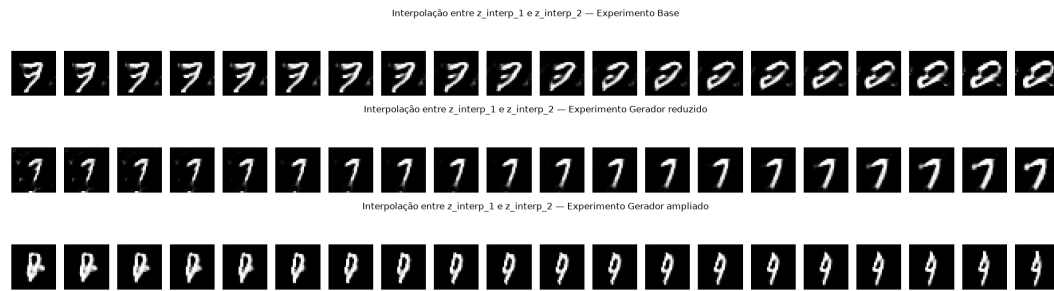
Nos experimentos com comportamento degradado, a evolução do vetor fixo evidencia de forma mais clara a dificuldade de aprendizagem. No gerador reduzido, a trajetória converge para uma forma pouco diversa e consistente com o colapso de modos observado na grade final. Já nos casos com discriminador ampliado e com ambas as redes ampliadas, não ocorre formação progressiva de uma estrutura de dígito claramente reconhecível, permanecendo padrões visualmente próximos de ruído durante grande parte do treinamento.

O experimento “LR gerador << discriminador” também apresenta evolução limitada, coerente com a dominância do discriminador observada nas curvas de perda. Em contraste, as configurações mais equilibradas apresentam transformação gradual da saída ao longo das épocas, indicando que o gerador conseguiu incorporar progressivamente a estrutura dos dados.



2.2.4 Interpolação no espaço latente

Para a análise da interpolação foram selecionadas três configurações: o modelo base, mantido como referência; o gerador ampliado, escolhido como exemplo de treinamento qualitativamente bem-sucedido; e o gerador reduzido, escolhido como exemplo de baixa qualidade. E evitamos utilizar um caso completamente degenerado em ruído. A seleção considerou conjuntamente a inspeção visual das amostras, o comportamento das perdas e os **scores** médios do discriminador.



No modelo base, a sequência mostra uma transformação contínua entre duas regiões distintas do espaço latente. As imagens iniciais possuem características próximas ao dígito 7 e, ao longo da interpolação, a estrutura vai sendo deformada gradualmente até assumir uma forma semelhante a 0. As imagens intermediárias permanecem coerentes com a trajetória entre os extremos, indicando que o gerador aprendeu uma representação latente relativamente suave.

No gerador ampliado, a interpolação também é contínua, mas ocorre entre formas visualmente mais próximas. A estrutura inicial apresenta uma região superior mais aberta e irregular e evolui progressivamente para uma forma mais estreita e alongada. Apesar de as diferenças entre os extremos serem menores que no modelo base, as mudanças entre passos sucessivos são suaves, sem rupturas bruscas ou aparecimento de ruído, reforçando a estabilidade qualitativa observada nesse experimento.

O gerador reduzido apresenta comportamento bastante diferente. Praticamente toda a sequência permanece restrita a variações do dígito 7. Há pequenas alterações na inclinação, espessura e curvatura dos traços, mas não ocorre transição para outra estrutura de dígito. Esse resultado é consistente com o **mode collapse** observado nas amostras finais: diferentes regiões do espaço latente são mapeadas para saídas muito semelhantes, reduzindo significativamente a diversidade representada pelo gerador.

2.2.5 Indicadores complementares de treinamento

Como complemento às curvas de perda, foram registrados os **scores** médios atribuídos pelo discriminador às amostras reais e falsas. Esses valores ajudam a interpretar o equilíbrio entre as duas redes: um discriminador dominante tende a produzir **scores** próximos de 1 para dados reais e de 0 para dados falsos, enquanto valores próximos de 0,5 para ambos indicam maior dificuldade de separação.

Experimento	Parâm.G	Parâm. D	Otimizador	LR G	LR D	Loss G final	Loss D final	Score reais final	Score falsos final
Base	840.321	212.865	RMSprop	10^{-3}	10^{-3}	1,4517	1,0734	0,6620	0,3387
Gerador reduzido	159.233	212.865	RMSprop	10^{-3}	10^{-3}	5,4494	0,2124	0,9500	0,0504
Gerador ampliado	3.567.105	212.865	RMSprop	10^{-3}	10^{-3}	0,7260	1,3988	0,5044	0,4958
Discriminador reduzido	840.321	42.225	RMSprop	10^{-3}	10^{-3}	1,0486	1,2052	0,5934	0,4064
Discriminador ampliado	840.321	906.561	RMSprop	10^{-3}	10^{-3}	0,0000	100,0000	1,0000	1,0000
Gerador e discriminador ampliados	3.567.105	906.561	RMSprop	10^{-3}	10^{-3}	0,0000	100,0000	1,0000	1,0000
LR gerador >> discriminador	840.321	212.865	RMSprop	10^{-3}	10^{-5}	0,6961	1,3829	0,5012	0,4989
LR gerador << discriminador	840.321	212.865	RMSprop	10^{-5}	10^{-3}	14,0794	0,1319	0,9742	0,0260
LR ambos menores	840.321	212.865	RMSprop	10^{-4}	10^{-4}	1,0220	1,1687	0,5944	0,4070
SGD	840.321	212.865	SGD	10^{-3}	10^{-3}	1,6274	0,7667	0,7298	0,2708

Na configuração base, os scores finais foram 0,6620 para amostras reais e 0,3387 para falsas, acompanhados de **Loss G = 1,4517** e **Loss D = 1,0734**. O gerador ampliado apresentou valores ainda mais próximos do equilíbrio, com **scores** de 0,5044 e 0,4958 e perdas finais de 0,7260 para o gerador e 1,3988 para o discriminador. O discriminador reduzido também permaneceu em uma região intermediária, com scores de 0,5934 para reais e 0,4064 para falsas.

O gerador reduzido apresenta o comportamento oposto: o discriminador atribui, em média, 0,9500 às amostras reais e apenas 0,0504 às falsas. Esse forte poder de separação acompanha **Loss G = 5,4494** e **Loss D = 0,2124** e é consistente com o **mode collapse** observado visualmente. Situação semelhante porém ainda mais acentuada ocorre quando “LR gerador << discriminador”, com **scores** de 0,9742 e 0,0260 e perda do gerador de 14,0794, indicando clara dominância do discriminador.

Nos experimentos com discriminador ampliado e com ambas as redes ampliadas, os resultados atingem um regime degenerado: os **scores** de reais e falsos chegam simultaneamente a 1,0000, enquanto **Loss G = 0** e **Loss D = 100**. Esses valores, associados às imagens semelhantes a ruído, mostram de forma particularmente clara que as perdas não podem ser interpretadas isoladamente como medida de qualidade do modelo.

Entre as variações restantes, “LR gerador >> discriminador” termina com **scores** praticamente iguais, 0,5012 para reais e 0,4989 para falsas, enquanto “LR ambos menores” apresenta 0,5944 e 0,4070. Com SGD, a separação permanece mais favorável ao discriminador, com 0,7298 para reais e 0,2708 para falsas. Em conjunto, esses indicadores complementam a inspeção visual e as curvas de perda, permitindo distinguir situações de equilíbrio adversarial, dominância do discriminador e falha degenerada do treinamento.

2.3 – Análise

A redução da capacidade do gerador produziu o efeito mais claro de **mode collapse**, com geração concentrada em formas semelhantes ao dígito 7, perda do gerador elevada e discriminador com forte capacidade de separação entre amostras reais e falsas. O aumento da capacidade do gerador, por outro lado, manteve boa diversidade visual e apresentou comportamento adversarial mais equilibrado. As alterações no discriminador foram mais críticas: o discriminador reduzido ainda permitiu treinamento estável, enquanto o discriminador ampliado e a configuração com ambas as redes ampliadas evoluíram para um regime degenerado, com imagens semelhantes a ruído e perdas que, isoladamente, poderiam levar a interpretações incorretas.

O desbalanceamento entre as taxas de aprendizado também teve impacto significativo. Quando “LR gerador << discriminador”, o discriminador passou a dominar o treinamento, produzindo **scores** próximos de 1 para imagens reais e de 0 para falsas, perda elevada do gerador e amostras degradadas. As configurações com “LR gerador >> discriminador” e com ambas as taxas reduzidas apresentaram comportamento mais estável e resultados visuais superiores.

No caso das taxas reduzidas, porém, deve-se considerar que atualizações menores dos parâmetros podem exigir mais iterações para atingir um nível comparável de convergência. Como todos os experimentos foram limitados às mesmas 10 épocas para preservar a comparabilidade, é possível que essas configurações ainda não tivessem atingido seu melhor desempenho ao final do treinamento. Um número maior de épocas poderia esclarecer se parte do comportamento observado decorre apenas de convergência mais lenta, embora isso não possa ser concluído a partir dos resultados atuais.

A substituição de RMSprop por SGD também permitiu aprendizagem, embora com menor equilíbrio entre gerador e discriminador em relação às melhores configurações com RMSprop.

Em conjunto, os resultados mostram que maior capacidade não implica necessariamente melhor treinamento e que o equilíbrio entre gerador e discriminador é mais importante do que a minimização isolada de qualquer uma das perdas. A análise conjunta das curvas, dos **scores**, das amostras finais, da evolução com vetor latente fixo e das interpolações foi necessária para distinguir treinamento estável, dominância do discriminador, **mode collapse** e falhas degeneradas.

Seção 3 – WGAN Condicional

3.1 – Setup

Foi utilizado o conjunto **Fashion-MNIST**, composto por imagens em escala de cinza de dimensão 28×28 , distribuídas em 10 classes de vestuário e acessórios. Para o treinamento foi utilizado o conjunto de treino, com lotes de **64 imagens**. As imagens foram convertidas para tensores e, em seguida, normalizadas do intervalo $[0, 1]$ para $[-1, 1]$ por meio da transformação $x' = 2x - 1$. Essa faixa é compatível com a ativação **Tanh** utilizada na saída do gerador.

O **gerador condicional** recebe um vetor latente z de dimensão 100 e o rótulo da classe desejada. O rótulo é convertido para uma representação **one-hot de 10 posições** e concatenado ao vetor latente, resultando em uma entrada de 110 dimensões. Essa entrada passa por uma camada totalmente conectada que produz $7 \times 7 \times 128$ valores, seguida por reorganização espacial, normalização em lote e duas convoluções transpostas até a geração de uma imagem $1 \times 28 \times 28$. A última camada utiliza **Tanh**.

O **crítico condicional** avalia conjuntamente a imagem e o rótulo. O vetor one-hot da classe é expandido espacialmente, produzindo 10 canais 28×28 , cada um preenchido pelo valor correspondente no vetor de classe. Esses canais são concatenados ao canal da imagem, produzindo uma entrada de **11 canais**. O crítico é composto por duas camadas convolucionais, com 64 e 128 filtros, ativações **LeakyReLU** e **Dropout**, seguidas por achatamento e uma camada linear que produz um único escore.

Foram mantidos os hiperparâmetros definidos no código-base: dimensão latente `z_dim=100`, tamanho de lote `batch_size=64`, `n_critic=5`, `clip_value=0.01`, otimizador **RMSprop** com taxa de aprendizado 10^{-4} para o gerador e para o crítico e treinamento por **100 épocas**. As perdas utilizadas foram:

$$L_C = \text{mean}(C(x_{fake}, c)) - \text{mean}(C(x_{real}, c))$$

para o crítico, e

$$L_G = -\text{mean}(C(x_{fake}, c))$$

para o gerador. Após cada atualização do crítico, seus pesos foram limitados ao intervalo $[-0.01, 0.01]$. Não foram alterados os hiperparâmetros solicitados no enunciado. Foi utilizada **seed 42** para melhorar a reprodutibilidade das execuções.

3.2 – Resultados

3.2.1 Curvas de perda

A figura a seguir apresenta as perdas médias do crítico e do gerador ao longo das 100 épocas. Como as duas perdas apresentam escalas distintas, elas foram exibidas separadamente, compartilhando o mesmo eixo de épocas.

Curvas de perda da WGAN condicional

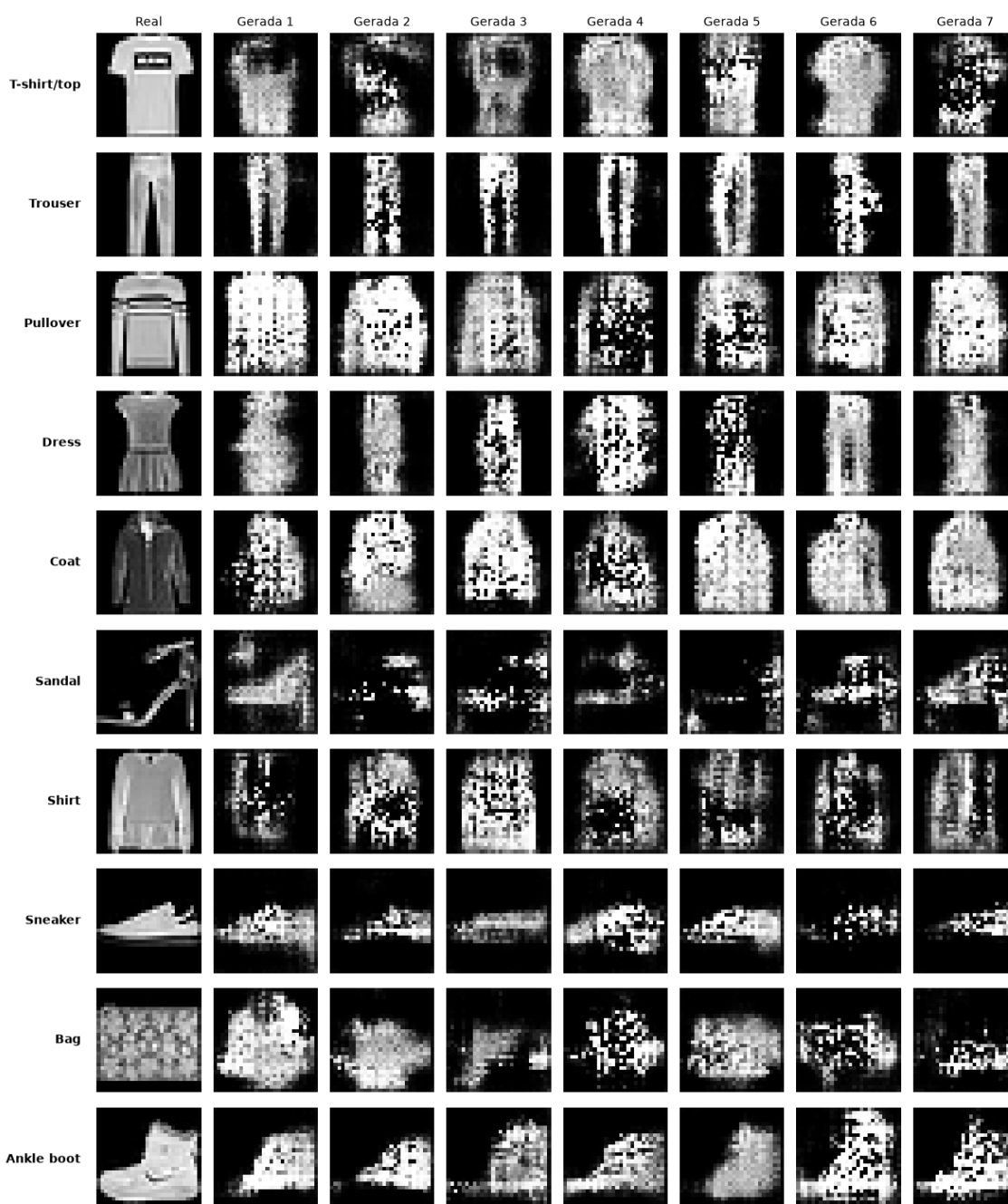


A perda do crítico apresentou uma redução acentuada nas primeiras épocas e posteriormente permaneceu próxima de zero. A perda do gerador apresentou oscilações maiores durante todo o treinamento, comportamento compatível com a dinâmica adversarial entre as duas redes.

3.2.2 Amostras condicionadas por classe

A figura a seguir apresenta uma grade organizada com **uma classe por linha**. A primeira coluna contém uma imagem real da respectiva classe, utilizada como referência visual, enquanto as demais colunas apresentam diferentes imagens produzidas pelo gerador para o mesmo rótulo e diferentes vetores latentes.

Grade de amostras por classe: referencia real e imagens geradas

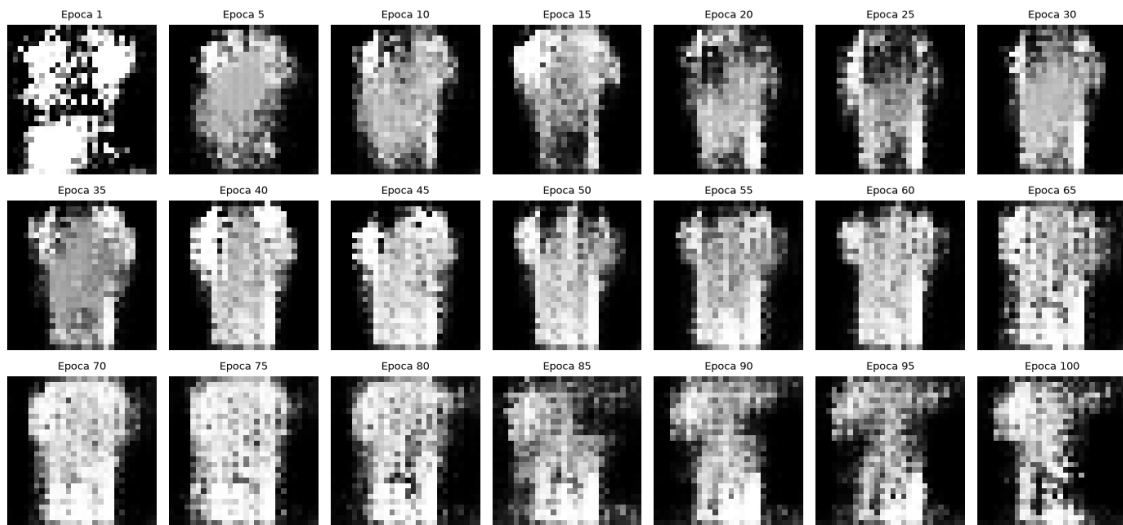


O condicionamento foi claramente observado em diversas classes, particularmente em *Trouser*, *Coat*, *Sneaker* e *Ankle boot*. As maiores ambiguidades ocorreram entre classes de vestuário visualmente semelhantes, como *T-shirt/top*, *Pullover* e *Shirt*.

3.2.3 Evolução de uma amostra durante o treinamento

Para acompanhar a evolução qualitativa do gerador, foram mantidos fixos tanto um vetor latente quanto o rótulo da classe *T-shirt/top*. A saída correspondente foi registrada na primeira época e, posteriormente, a cada cinco épocas até a época 100.

Evolução da geração para z fixo durante o treinamento - Classe: T-shirt/top



Nas primeiras épocas a saída apresenta pouca estrutura. Progressivamente surge uma silhueta compatível com a classe condicionada e, nas etapas intermediárias e finais, a forma da peça torna-se claramente reconhecível.

3.2.4 Interpolação entre classes com z fixo

No primeiro experimento de interpolação, um mesmo vetor latente foi mantido fixo enquanto o vetor de classe foi interpolado linearmente entre duas representações one-hot. Foram utilizadas as classes *Sneaker* e *Ankle boot*, que apresentaram boa qualidade de geração individual e permitem observar uma transformação visual natural entre os dois tipos de calçado.

Interpolacao de classes com z fixo: Sneaker -> Ankle boot



As imagens intermediárias mostram uma transformação gradual das características associadas ao rótulo, enquanto as demais características determinadas pelo vetor latente permanecem relativamente estáveis.

3.2.5 Interpolação no espaço latente com classe fixa

No segundo experimento, a classe foi mantida fixa enquanto dois vetores latentes z_1 e z_2 foram interpolados linearmente.

Interpolacao de z - classe fixa: Sneaker

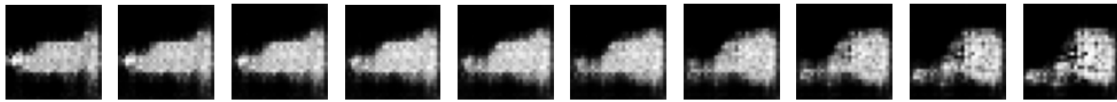


A sequência apresenta uma mudança contínua na forma e nas características da imagem, preservando a identidade geral da classe ao longo de toda a trajetória.

3.2.6 Interpolação simultânea do vetor latente z e da classe

No terceiro experimento, foram interpolados simultaneamente os vetores latentes $z_1 \rightarrow z_2$ e os vetores de classe $c_1 \rightarrow c_2$.

Interpolacao simultanea de z e classe: Sneaker -> Ankle boot

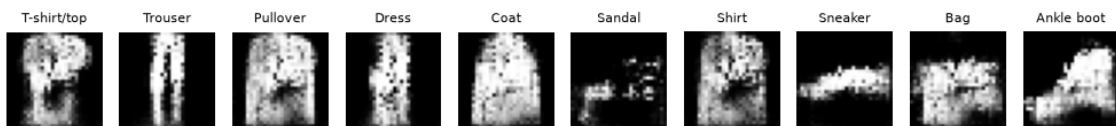


Nesse caso, tanto as características determinadas pelo espaço latente quanto aquelas determinadas pela condição de classe são modificadas progressivamente entre os dois extremos.

3.2.7 Mesmo vetor latente para as 10 classes

Por fim, foi utilizado um único vetor latente z , mantido fixo, variando-se exclusivamente o rótulo entre as 10 classes do Fashion-MNIST.

Mesmo vetor latente z com variacao da classe



Como o vetor latente permanece constante, as diferenças observadas entre as imagens resultam exclusivamente da alteração do rótulo fornecido ao gerador, permitindo verificar diretamente o efeito do condicionamento.

3.3 – Análise

O modelo foi capaz de gerar amostras reconhecíveis e de responder ao rótulo fornecido, demonstrando que o condicionamento foi efetivamente aprendido. Algumas classes, como *Trouser*, *Sneaker* e *Ankle boot*, apresentaram maior consistência, enquanto classes visualmente próximas, como *T-shirt/top*, *Pullover* e *Shirt*, apresentaram maior sobreposição.

A evolução de um vetor latente fixo mostrou a formação progressiva de uma estrutura compatível com a classe condicionada ao longo das épocas. As três interpolações produziram transições contínuas, indicando que tanto o espaço latente quanto a representação da classe foram incorporados de forma gradual pelo gerador.

A perda do crítico aproximou-se rapidamente de zero, indicando pequena diferença entre os escores médios atribuídos às amostras reais e geradas, embora esse comportamento também possa ser influenciado pelo `weight clipping` de 0,01. A perda do gerador apresentou oscilações ao longo do treinamento, comportamento esperado em um processo adversarial.

De forma geral, os resultados indicam que a WGAN condicional conseguiu aprender simultaneamente a estrutura do Fashion-MNIST e a informação fornecida pelos rótulos.

Seção 4 – Precisão e Revocação de Variedades

4.1 – Setup

Neste laboratório foram comparados três modelos generativos não condicionais — uma GAN totalmente conectada (GAN FC), uma DCGAN e uma WGAN convolucional — utilizando o conjunto Fashion-MNIST. As imagens possuem um canal e resolução de 28X28 pixels. Após a conversão para tensor, os valores dos pixels foram transformados da faixa [0,1] para [-1,1] pela transformação 2x-1, compatível com a ativação Tanh utilizada na saída dos geradores.

Para manter a comparação controlada, os três modelos utilizaram vetor latente de dimensão 100, amostrado de uma distribuição normal padrão, batch_size=64 e 50 épocas de treinamento. A GAN FC foi utilizada como configuração base. Seu gerador possui uma camada oculta com 256 neurônios e produz os 784 pixels da imagem por camadas totalmente conectadas. O discriminador também possui uma camada oculta com 256 neurônios, LeakyReLU, Dropout(0,5) e saída sigmoide.

A DCGAN utiliza um gerador convolucional comum à WGAN: o vetor latente é projetado inicialmente para 128 mapas de características de 7X7, seguido por BatchNorm e duas convoluções transpostas, com 64 canais intermediários, até a saída 1X28X28. O discriminador da DCGAN realiza o caminho inverso por duas convoluções com 64 e 128 canais, LeakyReLU com inclinação negativa 0,2, Dropout(0,4) e saída sigmoide.

A WGAN utiliza a mesma arquitetura de gerador da DCGAN e um crítico com a mesma estrutura convolucional básica do discriminador, porém sem sigmoide na saída. Além disso, emprega a função objetivo de Wasserstein, otimização com RMSprop, cinco atualizações do crítico para cada atualização do gerador e weight clipping de 0,01.

Modelo	Arquitetura	Capacidade principal	Perda	Otimizador	Learning Rate	n_critic	weight clipping	Diferenças em relação a configuração base
GAN FC	Totalmente conectada	hidden_size=256	BCE	Adam	0,001	1	—	Configuração base
DCGAN	Convolucional	hidden_channels=64 (64/128 canais)	BCE	Adam	0,001	1	—	Gerador/discriminador convolucionais, BatchNorm no gerador e Dropout(0,4) no discriminador
WGAN	Convolucional	hidden_channels=64 (64/128 canais)	Wasserstein	RMSprop	0,0001	5	0,01	Crítico sem sigmoide, objetivo Wasserstein, cinco atualizações do crítico por atualização do gerador e weight clipping

Para definir um espaço de características comum às três GANs, foi treinado um classificador auxiliar sobre o Fashion-MNIST. O extrator possui três camadas convolucionais com 32, 64 e 128 canais, intercaladas com ReLU e operações de pooling. Ao final, um AdaptiveAvgPool2d e um achatamento produzem um vetor de 128 características por imagem, utilizado posteriormente no cálculo das métricas MPR. A cabeça classificadora é uma camada linear de 128 para 10 classes.

O classificador foi treinado por 20 épocas, com CrossEntropyLoss, Adam com taxa de aprendizado 10^{-3} e batch_size de 128; a avaliação utilizou lotes de 512 imagens. Ao final da vigésima época, a acurácia no conjunto de teste foi de 90,2%. Esse extrator treinado foi mantido fixo e utilizado igualmente para os três modelos generativos.

4.2 – Resultados

4.2.1 Amostras Geradas

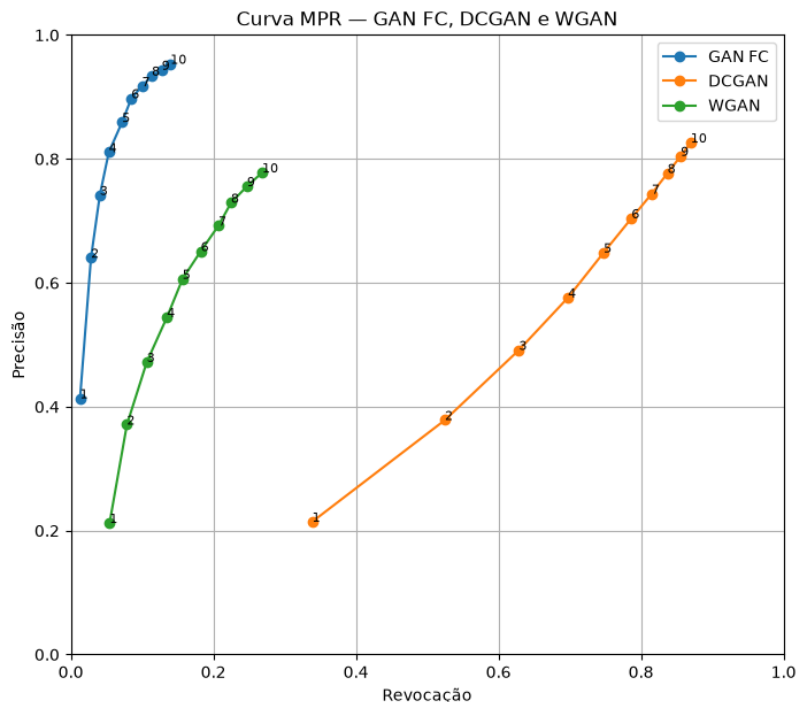
Foram geradas **25 amostras aleatórias por modelo**, organizadas em grades **5X5**, com o mesmo esquema de visualização e o mesmo conjunto de vetores latentes para os três geradores.



A inspeção visual mostra diferenças claras entre os modelos. A DCGAN apresenta, de forma geral, formas mais bem definidas e maior diversidade aparente de tipos de vestuário. A GAN FC gera diversas amostras reconhecíveis, mas também apresenta imagens com menor coerência espacial. A WGAN produz estruturas reconhecíveis, porém sua diversidade aparente é inferior à observada na DCGAN.

4.2.2 Curva MPR em função de K

Para a curva MPR foram utilizadas as **10.000 imagens do conjunto de teste** como aproximação da variedade real e **10.000 imagens geradas por cada modelo**. Todas as imagens foram representadas no mesmo espaço de 128 características produzido pelo classificador. O parâmetro k foi variado de 1 a 10.



Em todos os modelos, o aumento de k deslocou os pontos da curva para valores maiores de precisão e revocação. Isso ocorre porque um valor maior de k aumenta o raio local utilizado para aproximar cada variedade, tornando mais provável a inclusão de amostras da outra distribuição.

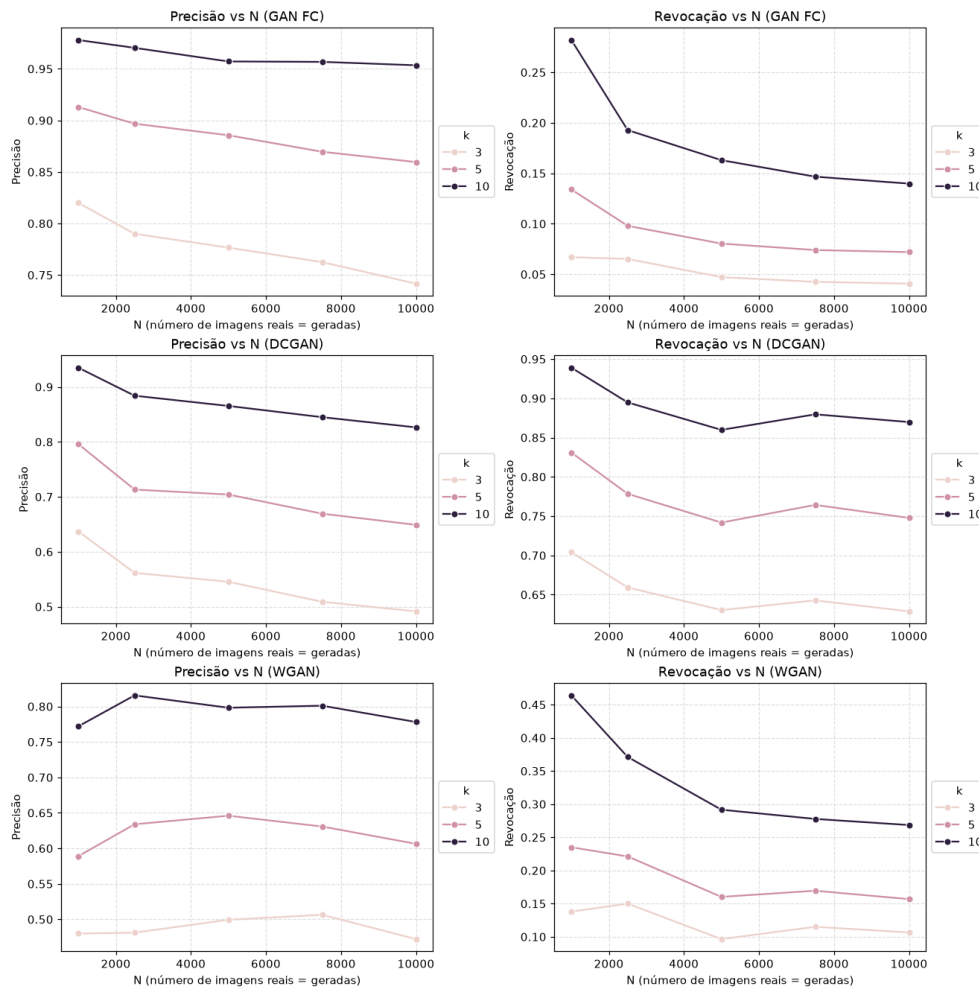
Como referência, os resultados para $N=10.000$ e os valores de k também utilizados na varredura em N foram:

K	GANFC		DCGAN		WGAN	
	Precisão	Revocação	Precisão	Revocação	Precisão	Revocação
3	0,7416	0,0407	0,4915	0,6287	0,4722	0,1066
5	0,8596	0,0720	0,6490	0,7477	0,6064	0,1568
10	0,9535	0,1398	0,8264	0,8698	0,7781	0,2685

A GAN FC obteve a maior precisão para os três valores apresentados, enquanto a DCGAN alcançou revocação muito superior aos demais modelos. A WGAN permaneceu em posição intermediária em revocação, mas ainda bastante abaixo da DCGAN.

4.2.3 Sensibilidade a N

A sensibilidade ao número de amostras foi avaliada com $N=1000, 2500, 5000, 7500, 10000$ e $k=3, 5, 10$, utilizando sempre a mesma quantidade de imagens reais e geradas.



Para a GAN FC, a precisão permaneceu elevada em todos os valores de N , enquanto a revocação diminuiu à medida que a aproximação da variedade passou a utilizar mais amostras. A DCGAN manteve revocação elevada e relativamente estável para os três valores de k , enquanto a WGAN apresentou revocação substancialmente inferior. De modo geral, as curvas passam a variar menos a partir de aproximadamente **$N=5000$** , tornando-se visualmente mais estáveis entre **$N=7500$** e **$N=10000$** .

4.3 – Análise

O aumento de k elevou a precisão e a revocação dos três modelos, pois amplia as vizinhanças utilizadas na aproximação das variedades. Para $N=10.000$, a ordenação observada em precisão foi **GAN FC > DCGAN > WGAN**, enquanto em revocação foi **DCGAN > WGAN > GAN FC**. A alta revocação da DCGAN concorda com sua grade de amostras, que apresenta maior diversidade visual e melhor cobertura das formas presentes no Fashion-MNIST. A GAN FC, por outro lado, combina precisão muito alta com revocação extremamente baixa, indicando que gera amostras plausíveis, mas concentradas em uma parcela restrita da variedade real, comportamento compatível com **colapso parcial de modos**.

A WGAN melhora a cobertura em relação à GAN FC, mas ainda apresenta revocação muito inferior à DCGAN. Vale ressaltar também que a tarefa da WGAN **não condicional** tende a ser mais difícil do que a de uma WGAN condicional equivalente, pois o treinamento não incorpora o vetor de classes, que fornece informação explícita sobre os modos da distribuição. Eventualmente, um número maior de épocas de treinamento poderia melhorar seus resultados, embora isso não seja garantido.

Na varredura de N , as curvas passam a apresentar variações menores a partir de cerca de 5.000 amostras e ficam mais estáveis entre 7.500 e 10.000. Assim, os resultados mostram que precisão alta isoladamente não implica boa cobertura da distribuição e que, neste experimento, a DCGAN apresentou o melhor equilíbrio entre fidelidade e diversidade.

Seção 5 – Modelos Autorregressivos com RNN

5.1 – Setup

Foram utilizadas receitas da base *Epicurious*, mantendo apenas registros com título e instruções disponíveis. Cada exemplo foi convertido para o formato “Recipe for <título> | <instruções>”, com todas as instruções da receita concatenadas em uma única sequência. Na normalização, os sinais de pontuação foram separados por espaços, sequências de múltiplos espaços foram reduzidas a um único espaço e todo o texto foi convertido para letras minúsculas. O vocabulário foi limitado aos **10.000 tokens mais frequentes**, incluindo os tokens especiais “<PAD>” e “<UNK>”. As **sequências de entrada foram limitadas a 200 tokens**, utilizando “padding” para sequências menores e truncamento para sequências maiores. O conjunto foi dividido de forma reprodutível **em 90% para treinamento e 10% para teste**, utilizando ‘SEED=42’, com **batch size de 32** exemplos.

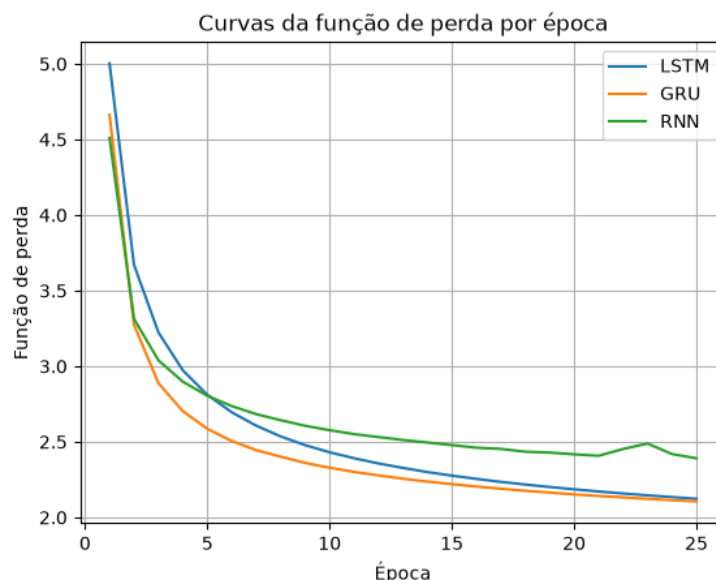
Em termos de arquitetura, foi montada a mesma estrutura geral: uma camada de “**embedding**” de **dimensão 100**, uma **camada recorrente com 128 unidades ocultas** e uma **camada linear que projeta o estado oculto para as 10.000 posições do vocabulário**. A única alteração entre os modelos foi o tipo da camada recorrente: “LSTM”, “GRU” ou “RNN clássica” com **ativação “tanh”**. O treinamento foi realizado por **25 épocas** com o **otimizador Adam**, taxa de aprendizado 10^{-3} e entropia cruzada como **função de perda (CrossEntropyLoss)**, ignorando o token “<PAD>”. A mesma semente, divisão dos dados e configuração de treinamento foram utilizadas nos três experimentos. A tabela a seguir resume os parâmetros utilizados:

Parâmetro	Valor	Descrição
VOCAB_SIZE	10.000	Tamanho máximo do vocabulário
MAX_LEN	200	Comprimento máximo da sequência de entrada
EMBEDDING_DIM	100	Dimensão do vetor denso de “embedding”
N_UNITS	128	Números de camadas recorrentes
BATCH_SIZE	32	Tamanho do lote de treinamento
EPOCHS	25	Número de épocas de treinamento
LEARNING_RATE	10^{-3}	Taxa de aprendizado do Adam
SEED	42	Semente aleatória para reprodutibilidade

5.2 – Resultados

5.2.1 Curvas de perda de treinamento

As curvas de perda da LSTM, GRU e RNN clássica foram registradas ao longo das 25 épocas e são apresentadas conjuntamente na figura a seguir:



Ao final do treinamento, as perdas foram aproximadamente: 2,1233 para a LSTM, 2,1056 para a GRU e 2,3912 para a RNN clássica, como pode ser observado na tabela:

Modelo	Perda no final do treinamento
LSTM	2,1233
GRU	2,1056
RNN	2,3912

5.2.2 Probabilidades no primeiro passo após o prompt

A geração foi iniciada, para cada um dos modelos, com o prompt “*recipe for roasted vegetables* |”. Foram utilizadas temperaturas 1,5, 1,0 e 0,2, além do modo guloso (**greedy**). A tabela a seguir apresenta as três palavras de maior probabilidade no primeiro passo de geração em cada configuração.

Modelo	Estratégia	Palavra escolhida	1ª palavra	Prob.	2ª palavra	Prob.	3ª palavra	Prob.
LSTM	Temp. 1,5	preheat	preheat	24,87%	in	6,30%	1	4,42%
LSTM	Temp. 1,0	preheat	preheat	60,39%	in	7,70%	1	4,53%
LSTM	Temp. 0,2	preheat	preheat	100,00%	in	0,00%	1	0,00%
LSTM	Greedy	preheat	preheat	60,39%	in	7,70%	1	4,53%
GRU	Temp. 1,5	preheat	preheat	7,44%	in	5,57%	place	3,76%
GRU	Temp. 1,0	whisk	preheat	17,20%	in	11,14%	place	6,19%
GRU	Temp. 0,2	preheat	preheat	88,20%	in	10,05%	place	0,53%
GRU	Greedy	preheat	preheat	17,20%	in	11,14%	place	6,19%
RNN	Temp. 1,5	preheat	preheat	7,13%	combine	4,89%	in	4,65%
RNN	Temp. 1,0	cut	preheat	16,68%	combine	9,46%	in	8,79%
RNN	Temp. 0,2	preheat	preheat	89,57%	combine	5,28%	in	3,64%
RNN	Greedy	preheat	preheat	16,68%	combine	9,46%	in	8,79%

A redução da temperatura tornou a distribuição mais concentrada já no primeiro passo. Esse comportamento é especialmente evidente na LSTM com temperatura 0,2, em que “*preheat*” recebeu praticamente toda a probabilidade. Em temperatura 1,0, entretanto, a GRU selecionou “*whisk*” e a RNN clássica selecionou “*cut*”, embora “*preheat*” fosse a palavra de maior probabilidade nos dois casos. Isso ocorre porque a geração por amostragem permite selecionar tokens diferentes do “*argmax*”. No modo *greedy*, por outro lado, o token de maior probabilidade é sempre escolhido, razão pela qual os três modelos iniciaram a geração com “*preheat*”.

5.2.3 Textos Gerados

Para avaliar qualitativamente o efeito da arquitetura recorrente e da temperatura de amostragem, foram gerados textos a partir do mesmo prompt, “*recipe for roasted vegetables* |”, com limite de 25 tokens. Para cada modelo foram utilizadas as temperaturas 1,5, 1,0 e 0,2, além do modo “*greedy*”. A tabela a seguir mostra os textos gerados.

Estratégia	LSTM	GRU	RNN Clássica
Greedy	recipe for roasted vegetables preheat oven to 350°f . butter and flour a 9 - inch square baking pan . in a large bowl	recipe for roasted vegetables preheat oven to 400°f . place garlic in large bowl . add garlic and next 5 ingredients ; toss to	recipe for roasted vegetables preheat oven to 350°f . butter and flour two 9 - inch - diameter springform pan with 2 - inch
Temp. 0,2	recipe for roasted vegetables preheat oven to 350°f . butter and flour heavy large baking sheet . place 1 / 2 cup water in	recipe for roasted vegetables preheat oven to 400°f . place garlic in large bowl . add garlic and garlic . mix in 1 /	recipe for roasted vegetables preheat oven to 350°f . butter and flour two 9 - inch - diameter cake pan with 2 - inch
Temp. 1,0	recipe for roasted vegetables preheat broiler . grill or 6 to 7 minutes per side down with a rack . cut fish into 4	recipe for roasted vegetables whisk buttermilk , wine , and 1 teaspoon lemon juice , port , and 1 / 2 teaspoon salt in	recipe for roasted vegetables cut swordfish into 5 - inch - wide square in the empty sand . 4 . season with salt and
Temp. 1,5	recipe for roasted vegetables preheat broiler and reserve for 8 . grate beets onto the crevices ; place in a baking dish and bake	recipe for roasted vegetables preheat chickens directly onto baking sheet . leave glaze rinse . we releasing any juices in dish with olive oil	recipe for roasted vegetables preheat 18 cups cold oil and heat syrup over moderately high tomatoes them , cook for 5 minutes , or

As gerações mostram que temperaturas mais baixas e o modo “greedy” tendem a produzir sequências mais previsíveis e estruturalmente coerentes, enquanto temperaturas mais altas aumentam a diversidade, mas também favorecem combinações menos plausíveis.

Além disso, mesmo quando a estrutura sintática se mantém razoável, observa-se baixa aderência semântica ao tema do prompt: receitas iniciadas como “*roasted vegetables*” passam a incluir ingredientes ou preparos pouco relacionados, como peixe, frango, sobremesas ou técnicas incompatíveis com a ideia original.

Esse desvio indica que os modelos aprenderam padrões locais de escrita de receitas, mas não necessariamente preservam o contexto semântico global ao longo da sequência. O efeito aparece nos três modelos, embora a perda de coerência e de aderência ao tema seja mais evidente na GRU e, principalmente, na RNN clássica nas amostras com temperatura 1,5.

5.2.4 Casos selecionados durante a geração

Além da análise do primeiro passo, foram selecionados automaticamente casos representativos ao longo das gerações. Para cada modelo foram considerados dois tipos de situação: um passo em que o token amostrado divergiu significativamente do token de maior probabilidade e um passo, com temperatura 0,2, em que a distribuição ficou fortemente concentrada em um único token.

Modelo	Estratégia	Passo	Situação	Token + provável	Prob.	Token Selecionado	Prob.
LSTM	Temp. 1,0	2	Amostragem divergente de argmax	oven	89,44%	broiler	8,03%
LSTM	Temp. 0,2	3	Distribuição concentrada	to	100,00%	to	100,00%
GRU	Temp. 1,0	11	Amostragem divergente de argmax	in	91,46%	, (o sinal de vírgula)	2,09%
GRU	Temp. 0,2	3	Distribuição concentrada	to	100,00%	to	100,00%
RNN	Temp. 1,5	12	Amostragem divergente de argmax	heat	92,66%	tomatoes	0,00%
RNN	Temp. 0,2	3	Distribuição concentrada	to	100,00%	to	100,00%

Na LSTM, após “*recipe for roasted vegetables | preheat*”, o token “oven” possuía probabilidade de 89,44%, mas a amostragem com temperatura 1,0 selecionou “broiler”, com 8,03%. Na GRU, após “*whisk buttermilk , wine , and 1 teaspoon lemon juice*”, o token mais provável era “in”, com 91,46%, enquanto o token selecionado foi a vírgula, com apenas 2,09%. O caso mais acentuado ocorreu na RNN clássica com temperatura 1,5: após “*preheat 18 cups cold oil and heat syrup over moderately high*”, “heat” possuía 92,66% de probabilidade, mas foi selecionado “tomatoes”, cuja probabilidade arredondada para duas casas foi 0,00%.

Os casos de distribuição concentrada foram semelhantes nos três modelos. Com temperatura 0,2, após a sequência “*preheat oven*”, o token “to” recebeu praticamente 100% da probabilidade e foi selecionado em todos eles. Esses exemplos evidenciam os dois efeitos principais da temperatura: valores baixos tornam a geração quase determinística, enquanto valores mais altos permitem a seleção de tokens muito menos prováveis, aumentando a diversidade, mas também o risco de perda de coerência.

5.2.5 NLL e perplexidade no conjunto de teste

A NLL foi calculada como a média por token válido, ignorando “<PAD>”, e a perplexidade foi obtida pela fórmula “ $\exp(NLL)$ ”. Os três modelos foram avaliados sobre o mesmo conjunto de teste. A tabela a seguir mostra os números obtidos

Modelo	NLL Média	Perplexidade
LSTM	2,3422	10,40
GRU	2,3433	10,42
RNN Clássica	2,5876	13,30

A LSTM apresentou a menor NLL média e, consequentemente, a menor perplexidade, com valores muito próximos aos obtidos pela GRU. Isso indica que as duas arquiteturas tiveram desempenho praticamente equivalente na previsão dos tokens do conjunto de teste. A RNN clássica, por outro lado, apresentou NLL e perplexidade claramente superiores, evidenciando

maior incerteza nas previsões. Como a perplexidade é calculada a partir da NLL média por token, as duas métricas preservam a mesma ordenação entre os modelos: LSTM, GRU e RNN clássica.

5.3 – Análise

A LSTM apresentou a menor perplexidade no conjunto de teste (10,40), praticamente empatada com a GRU (10,42), enquanto a RNN clássica obteve resultado claramente inferior (13,30). Essa ordenação é coerente com as curvas de treinamento, nas quais LSTM e GRU convergiram para perdas próximas e menores que a RNN, embora a GRU tenha terminado com perda de treinamento ligeiramente inferior à LSTM.

Nas gerações, temperaturas baixas e o modo “*greedy*” produziram sequências mais previsíveis e estruturalmente coerentes, enquanto temperaturas mais altas aumentaram a diversidade e a chance de selecionar tokens pouco prováveis, como observado nos casos selecionados durante a geração.

Entretanto, mesmo nas saídas sintaticamente plausíveis, os três modelos apresentaram limitações de aderência semântica ao prompt, introduzindo ingredientes e preparos pouco relacionados a “*roasted vegetables*”. Esse comportamento sugere que os modelos aprenderam bem padrões locais de linguagem de receitas, mas têm dificuldade em preservar o contexto semântico global ao longo da sequência. A perda de coerência foi mais evidente em temperaturas altas, especialmente na RNN clássica, enquanto LSTM e GRU apresentaram comportamento mais estável.

Em conjunto, os resultados sugerem que os mecanismos de memória e controle presentes na LSTM e na GRU favorecem a modelagem de dependências mais longas em comparação com a RNN clássica, embora não sejam suficientes, neste experimento, para garantir consistência semântica global. Esse resultado está em linha com as limitações discutidas em sala de aula para esse tipo de modelo.

Seção 6 – Mini GPT

6.1 – Setup

6.1.1 Dados e Hiperparâmetros de Treinamento

Foram utilizados exatamente os mesmos dados do **Laboratório 5**, provenientes do conjunto ***Epicurious***. As receitas permaneceram no formato “*recipe for <título> | <instruções>*”, reaproveitando a mesma filtragem, normalização e construção do vocabulário. Além disso, o **Laboratório 6 reutilizou diretamente** o mesmo objeto de **dataset**, os mesmos **subconjuntos de treinamento e teste produzidos pelo *split*** e os mesmos **“DataLoaders”** (variáveis “train_dl” e “test_dl”). Dessa forma, “VOCAB_SIZE=10.000”, “MAX_LEN=200”, “BATCH_SIZE=32” e a divisão dos dados foram preservados sem qualquer nova amostragem ou redistribuição entre treino e teste. Isso garante que as diferenças observadas entre MiniGPT, LSTM, GRU e RNN sejam atribuídas aos modelos e configurações experimentais, e não a variações nos dados avaliados.

Todos os experimentos do Laboratório 6 foram treinados por 25 épocas, utilizando “SEED=42” para reprodutibilidade. A otimização foi realizada com Adam e taxa de aprendizado 10^{-3} . A função de perda utilizada foi a NLL calculada sobre os tokens válidos da sequência, ignorando o token de “padding”. A avaliação no conjunto de teste seguiu o mesmo critério, utilizando a NLL média por token para o cálculo da perplexidade. Como todos os modelos foram avaliados com o mesmo vocabulário, o mesmo “test_dl” e o mesmo procedimento de cálculo, as perplexidades obtidas podem ser comparadas diretamente com as dos modelos recorrentes do Laboratório 5.

6.1.2 Arquitetura do MiniGPT

A configuração base do MiniGPT foi composta por uma camada de “**embedding**” de tokens combinada com “**embedding**” posicional, seguida por **um único bloco Transformador** com atenção causal “*multi-head*” e uma rede “**feed-forward**”. Foi aplicada a máscara causal, que impede que cada posição utilize informações de tokens futuros durante o treinamento, preservando o caráter auto-regressivo do modelo. A saída do bloco Transformador é projetada para o tamanho do vocabulário por uma camada linear e convertida em log-probabilidades com “*LogSoftmax*”, utilizadas posteriormente pela “*NLLLoss*”.

6.1.3 Configurações Experimentais

A configuração base foi utilizada como referência para todos os experimentos. A partir dela, cada hiperparâmetro do bloco Transformador foi variado isoladamente, mantendo os demais inalterados. Foram testados valores menores e maiores para a dimensão do “**embedding**”, o número de cabeças de atenção e a dimensão da rede “**feed-forward**”. Em todos os casos, um novo modelo e um novo otimizador foram instanciados após a execução de `torch.manual_seed(SEED)`, preservando as demais condições de treinamento.

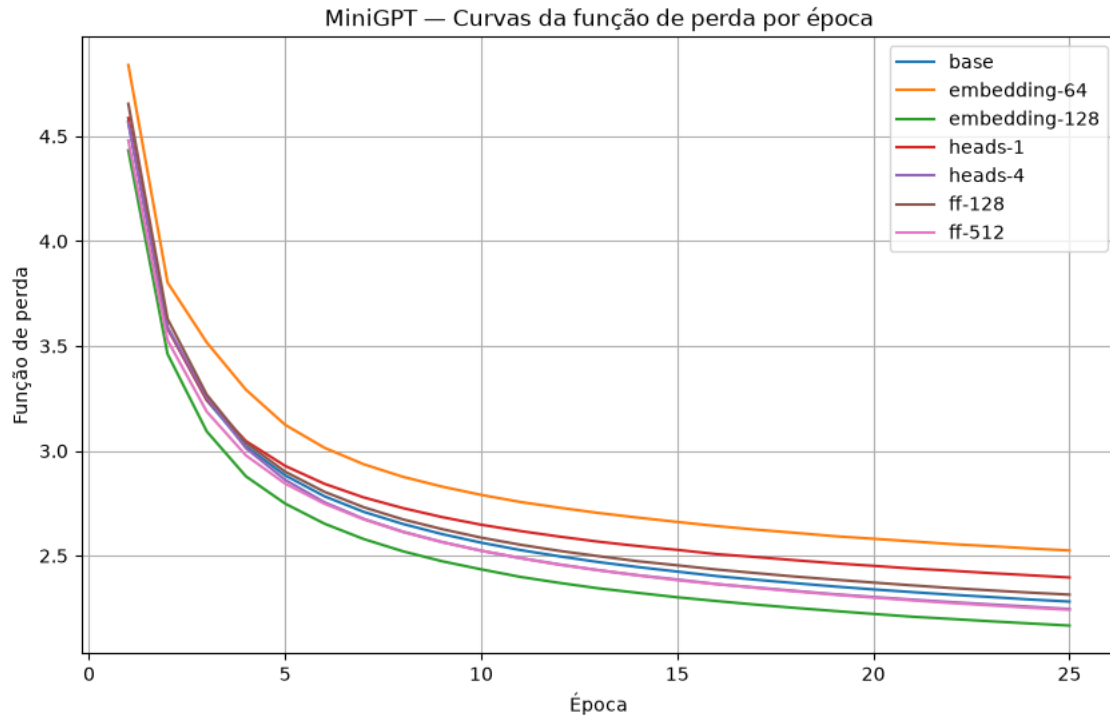
Experimento	EMBED	HEADS	FF_DIM	Descrição	NNL	PPL	N. Param.
base	100	2	256	Referência	2,3841	10,85	2.122.356
embedding-64	64	2	256	Redução da dimensão do “embedding”	2,5663	13,02	1.352.784
embedding-128	128	2	256	Aumento da dimensão do “embedding”	2,3146	10,12	2.728.080
heads-1	100	1	256	Redução do número de cabeças de atenção	2,4761	11,90	2.122.356
heads-4	100	4	256	Aumento do número de cabeças de atenção	2,3670	10,67	2.122.356
ff-128	100	2	128	Redução da dimensão da rede “feed-forward”	2,4198	11,24	2.096.628
ff-512	100	2	512	Aumento da dimensão da rede “feed-forward”	2,3446	10,43	2.173.812

Nos experimentos com “N_HEADS”, o “**embedding**” foi mantido em 100 para preservar a divisibilidade entre as cabeças. Cada teste alterou apenas um hiperparâmetro da configuração base, isolando seu efeito no desempenho.

6.2 – Resultados

6.2.1 Curvas de Perda de Treinamento

As curvas de perda dos sete experimentos MiniGPT foram acompanhadas ao longo das 25 épocas de treinamento e são apresentadas conjuntamente na figura abaixo.



As curvas mostram comportamento consistente com os resultados de perplexidade obtidos posteriormente no conjunto de teste. A configuração “embedding-128” apresentou as menores perdas ao final do treinamento, enquanto “embedding-64” permaneceu com valores mais elevados. As demais configurações ficaram em uma faixa intermediária, com diferenças menos acentuadas entre si. De forma geral, o gráfico indica que a dimensão do **“embedding”** foi a variação que produziu o maior impacto no comportamento de convergência entre os experimentos.

6.2.2 Efeito dos Hiperparâmetros sobre NLL e Perplexidade

A variação da dimensão do “*embedding*” produziu o maior impacto sobre a perplexidade. Em relação à configuração base (PPL=10,85), a redução de “EMBEDDING_DIM” para 64 elevou a perplexidade para 13,02, enquanto o aumento para 128 reduziu a métrica para 10,12, o melhor resultado entre todos os experimentos.

A alteração do número de cabeças de atenção também apresentou efeito mensurável, porém menos intenso. Com uma única cabeça, a perplexidade aumentou para 11,90, enquanto quatro cabeças reduziram o valor para 10,67. Como “EMBEDDING_DIM” permaneceu em 100, essas alterações não modificaram o número total de parâmetros do modelo.

Na dimensão da rede “*feed-forward*”, reduzir “FEED_FORWARD_DIM” de 256 para 128 elevou a perplexidade para 11,24. Já o aumento para 512 reduziu a perplexidade para 10,43. Esse ganho foi inferior ao observado com “embedding-128”, embora tenha exigido um aumento relativamente pequeno no número de parâmetros.

No conjunto dos experimentos, os resultados indicam que a **dimensão do “embedding”** foi o hiperparâmetro com **maior influência sobre o desempenho** do MiniGPT, seguida pelo número de cabeças de atenção e pela dimensão da camada “*feed-forward*”.

6.2.3 Textos Gerados pelo MiniGPT base

Foram gerados textos com a configuração base do MiniGPT a partir do prompt “recipe for roasted vegetables |”, utilizando as temperaturas 0,1, 0,5, 1,0 e 1,5. Em todos os casos, a geração foi limitada a 25 tokens.

Configuração	Frase Gerada
Temperatura 0,1	recipe for roasted vegetables preheat oven to 450°f . in a large heavy skillet heat oil over moderately high heat until hot but not
Temperatura 0,5	recipe for roasted vegetables preheat oven to 450°f . generously butter and butter a 9 - inch skillet over moderate heat until foam subsides
Temperatura 1,0	recipe for roasted vegetables cut red bell pepper on a rack in a broiler and add onion (season with salt and pepper .
Temperatura 1,5	recipe for roasted vegetables cut red bell pepper on an cauliflower in half lengthwise and enclose . scoop left lobster . using a chopstick

As gerações evidenciam o efeito da temperatura sobre a distribuição de amostragem. Com temperatura 0,1, o texto é mais conservador e mantém uma estrutura culinária coerente, enquanto 0,5 introduz maior variação sem perder completamente a plausibilidade. Em 1,0, a diversidade aumenta e já surgem construções menos naturais. Com temperatura 1,5, a perda de coerência sintática e semântica torna-se mais evidente, incluindo combinações pouco plausíveis e elementos sem aderência ao tema de “roasted vegetables”, como a introdução de “lobster”.

6.2.4 Comparação entre o melhor MiniGPT e as RNNs

O experimento “embedding-128” apresentou a menor perplexidade entre as configurações MiniGPT e foi selecionado para comparação com os modelos recorrentes do Laboratório 5. Como todos foram avaliados sobre o mesmo conjunto de teste, com o mesmo vocabulário e com a NLL média calculada por token não “padding”, os valores de perplexidade podem ser comparados diretamente.

Modelo	NLL	PPL	Parâmetros
MiniGPT (embedding-128)	2,3146	10,12	2.728.080
LSTM	2,3420	10,40	2.407.760
GRU	2,3433	10,42	2.378.320
RNN	2,5887	13,31	2.319.440

O MiniGPT “embedding-128” obteve a menor NLL e a menor perplexidade da comparação, superando ligeiramente LSTM e GRU e apresentando vantagem mais clara sobre a RNN clássica. Esse ganho, porém, veio acompanhado do maior número de parâmetros entre os modelos comparados. Assim, o resultado indica melhor desempenho preditivo para essa configuração do MiniGPT, mas com custo adicional de capacidade do modelo, aspecto que deve ser considerado na análise final.

6.3 – Análise

A variação de “EMBEDDING_DIM” foi a que mais afetou a perplexidade do MiniGPT: a redução para 64 elevou a PPL para 13,02, enquanto o aumento para 128 produziu o melhor resultado, com PPL de 10,12. As alterações em “N_HEADS” e “FEED_FORWARD_DIM” também influenciaram o desempenho, mas com impacto menor.

Nas gerações do MiniGPT base, temperaturas baixas produziram textos mais previsíveis e coerentes, enquanto o aumento da temperatura ampliou a diversidade ao custo de maior perda de coerência sintática e semântica, chegando a introduzir elementos pouco relacionados ao prompt “roasted vegetables”.

Na comparação quantitativa, o MiniGPT “embedding-128” apresentou a menor perplexidade (10,12), superando LSTM (10,40), GRU (10,42) e RNN clássica (13,31). Esse ganho, porém, veio com o maior número de parâmetros, 2.728.080, contra cerca de 2,4 milhões nos modelos recorrentes, indicando melhor desempenho preditivo ao custo de maior capacidade.

Como apenas a configuração base do MiniGPT foi utilizada nas gerações de texto, os resultados não permitem afirmar que o “embedding-128” também produza textos qualitativamente superiores aos das RNNs.